Table des matières

Cours 01 — Analyser les sources 1
Ce que vous allez faire 1
Avant de commencer 2
Étape 1 — Ce que le café a livré · Sources 2
Étape 2 — Lire un fichier sans le charger · Sources 3
Étape 3 — Le jour exporté deux fois · Sources 4
Étape 4 — Le torréfacteur · Sources 5
À retenir 6

Cours 01 — Analyser les sources

Cours 1/12 · La Brûlerie — Formation Data Engineering

Sources	**Ingestion**	**RAW**
d'où vient la donnée	aller la chercher, et la charger	la déposer, la ranger, la traiter
✓ ici		

Ce que vous allez faire

La Brûlerie du Passage, un café de quartier fictif à Montréal, vous confie un mois de fichiers : les exports de sa caisse et les traces de son torréfacteur. Personne ne les a jamais chargés nulle part. Avant d'écrire une ligne de pipeline, un ingénieur de données regarde ce qu'on lui a livré : combien de fichiers, quelles colonnes, quels types, et ce qui cloche.

fichiers livrés → DuckDB les lit là où ils sont → deux constats qui décident du pipeline

Les deux constats, que vous allez trouver vous-même : la caisse a exporté un jour deux fois, et le journal des lots du torréfacteur connaît tout le mois avant que les courbes arrivent. Les cours suivants bâtissent le pipeline autour de ces deux faits.

Rien ne se charge dans ce cours, et aucun fichier du projet ne s'écrit. Chaque requête se donne en pseudocode : les étapes en commentaire, et des trous ____ à remplir. Le code complet est dans `Codes_01_DE.md`, à ouvrir après avoir essayé. Le café et ce qu'on attend de vous sont présentés dans `Mise_en_situation_La_Brulerie_01_DE.md`, dans le dossier `01_Brulerie_facile`.

© 2026 iAlexMG. Tous droits réservés.

Avant de commencer

Le cours 00 a mis le mois livré dans votre projet : si `Projet_Brulerie\sources` manque, faites-le d'abord. Deux fenêtres servent aux cours : l'Anaconda Prompt pour les commandes, et le notebook de DuckDB, dans le navigateur, pour les requêtes SQL (il s'ouvre à l'étape 2). Placez l'Anaconda Prompt dans `Data_Domain`, le dossier du parcours :

```
conda activate formations
REM Attendu : (formations) au début de l'invite.
cd Projet_Brulerie
dir /b
REM Attendu : sources, plus ce qu'Analytics Engineering y a laissé, si vous l'avez fait.
```

L'Anaconda Prompt reste dans `Projet_Brulerie` jusqu'à la fin du cours, et chaque cours suivant s'y ouvre.

Étape 1 — Ce que le café a livré · Sources

La caisse exporte chaque soir deux fichiers, les tickets et leurs lignes. Le torréfacteur tient un journal de ses lots et écrit une courbe de température par jour de torréfaction. Le dictionnaire dit ce que veut dire chaque colonne : c'est la première lecture d'un ingénieur de données.

1.1 Lister ce qui est livré.

```
dir sources
REM Attendu : sept dossiers, dont caisse et torrefaction, et trois fichiers, dont dictionnaire_brulerie.md.
```

Les cours emploient surtout `caisse` et `torrefaction`, parce que ce sont les deux sources qui arrivent jour après jour, comme les livraisons qu'un pipeline prend chaque matin. Chacune cache un défaut que les cours apprennent à traiter. La météo (`meteo`) sert au cours 04, qui la tire aussi de son API, puis à Data Science ; les autres dossiers servent aux formations qui suivent, surtout Analytics Engineering (`catalogue`, `fidelite`) et AI Engineering (`documents`). Le cours 03 charge quand même le catalogue et la fidélité dans la `RAW`, tels quels, pour Analytics Engineering.

1.2 Lister les fichiers de la caisse d'un seul jour, le 1er juin. `dir /b` donne les noms seuls, sans date ni taille, et le joker `*` remplace n'importe quelle suite de caractères.

```
dir /b sources\caisse\2025-06\*_2025-06-01_*
REM Attendu : deux fichiers, lignes_2025-06-01_202506011707.csv et tickets_2025-06-01_202506011707.csv.
```

Un export donne deux fichiers : `tickets_…`, un ticket par ligne, et `lignes_…`, un produit vendu par ligne. Le nom porte le jour (`2025-06-01`), puis l'instant de l'export (`202506011707`, le 1er juin à 17 h 07).

1.3 Compter les exports de tickets du mois. La barre `|` passe la liste des noms à `find /c ".csv"`, qui compte les lignes contenant `.csv`, donc les fichiers.

```
dir /b sources\caisse\2025-06\tickets_* | find /c ".csv"
REM Attendu : 31.
```


© 2026 iAlexMG. Tous droits réservés.

Juin compte 30 jours : même si le café ouvrait tous les jours, 31 exports, c'est un de trop. Un jour a donc été exporté deux fois. L'étape 3 trouve lequel.

1.4 Lister les courbes du torréfacteur. Ici, pas besoin de compter : huit noms se lisent d'un coup d'œil, et ce sont leurs dates qui comptent.

```
dir /b sources\torrefaction\courbes\2025-06
REM Attendu : 8 fichiers, du courbe_2025-06-03.csv au courbe_2025-06-27.csv.
```

Une courbe par jour de torréfaction : le 3, le 6, le 10, le 13 juin, et ainsi de suite. Le torréfacteur rôtit le mardi et le vendredi, huit jours en juin.

1.5 Lire les sections 1 (la caisse) et 4 (le torréfacteur) du dictionnaire.

```
notepad sources\dictionnaire_brulerie.md
```

Fermez le Bloc-notes avant de passer à l'étape 2.

Étape 2 — Lire un fichier sans le charger · Sources

DuckDB lit un fichier CSV là où il est : `read_csv` prend la place d'un nom de table, et rien ne se copie. Lancé sans nom de fichier, DuckDB travaille en mémoire : aucun entrepôt ne s'écrit sur le disque. Un joker `*` dans le chemin lit plusieurs fichiers d'un coup, comme une seule table.

2.1 Ouvrir le notebook de DuckDB, depuis l'Anaconda Prompt.

```
duckdb -ui
```

- L'interface s'ouvre dans le navigateur, à `http://localhost:4213`. Au premier lancement, DuckDB télécharge son extension `ui` : il faut Internet.
- Laisser l'Anaconda Prompt ouvert : le fermer arrête l'interface.
- Créer un notebook (`+` à côté de Notebooks) et le renommer « Cours 01 » (double-clic sur son titre, puis Entrée) : chaque cours aura le sien.
- Une requête par cellule (`Add Cell`). Ctrl+Entrée l'exécute. Les lignes `--` sont des commentaires : DuckDB les ignore.

Les requêtes qui suivent se tapent dans ce notebook, jusqu'à la sous-étape 4.4.

2.2 Lire les premiers tickets du 1er juin. Le nom d'un export porte son jour, puis l'instant de l'export : le joker remplace cet instant.

```
-- 1. quatre colonnes des cinq premiers tickets du 1er juin
SELECT ticket_id, horodatage, statut, total
FROM read_csv('sources/caisse/2025-06/____')
LIMIT 5;
-- Attendu : 5 lignes, du ticket 158413 (8 h 06) au ticket 158417.
```

2.3 Demander à DuckDB les types qu'il devine, sur tous les exports du mois.

© 2026 iAlexMG. Tous droits réservés.

```
-- 1. les colonnes et les types devinés, tous les fichiers de tickets confondus
DESCRIBE SELECT * FROM read_csv('sources/caisse/*/____');
-- Attendu : 13 colonnes ; sous_total, tps, tvq, total, arrondi et encaisse en DOUBLE.
```

Un `DOUBLE` est un nombre à virgule flottante : une approximation en binaire, où un montant comme 0,10 ne tombe jamais juste. Pour de l'argent compté au cent, c'est le mauvais type.

2.4 Additionner l'encaissé du mois, puis relancer la même cellule deux ou trois fois (Ctrl+Entrée), pour voir que le même calcul sur les mêmes fichiers ne rend pas toujours le même nombre.

```
-- 1. la somme de l'encaissé, tous les exports du mois
SELECT SUM(____) AS encaisse
FROM read_csv('sources/caisse/*/tickets_*.csv');
-- Attendu : presque 19352.36, par exemple 19352.359999999993 ou 19352.359999999997,
-- et parfois 19352.36 tout rond : les derniers chiffres changent d'une fois à l'autre.
```

DuckDB additionne sur plusieurs fils, dans un ordre qui change d'une exécution à l'autre, et un `DOUBLE` ne donne pas tout à fait le même résultat dans un autre ordre. Un total qui bouge sans que les données bougent ne se compare à rien : le cours 02 déclare les montants en `DECIMAL`. Ce total est d'ailleurs faux, pour une autre raison, que l'étape 3 montre.

Étape 3 — Le jour exporté deux fois · Sources

Un numéro de ticket ne sert qu'une fois : la caisse les donne dans l'ordre, depuis son installation. Compter les lignes et les numéros distincts dit donc si un ticket revient.

3.1 Compter les lignes et les tickets distincts du mois.

```
-- 1. les lignes, puis les numéros de tickets distincts
SELECT COUNT(*) AS lignes, COUNT(DISTINCT ____) AS tickets
FROM read_csv('sources/caisse/*/tickets_*.csv');
-- Attendu : 1585 lignes pour 1527 tickets.
```

58 lignes de trop : 58 tickets reviennent deux fois. Il reste à trouver d'où. L'option `filename = true` ajoute à chaque ligne le nom du fichier d'où elle vient, et `REGEXP_EXTRACT` en tire le jour : la partie entre parenthèses du motif.

3.2 Trouver le jour exporté plus d'une fois.

```
-- 1. tirer le jour du nom de chaque fichier
SELECT REGEXP_EXTRACT(filename, 'tickets_(\d{4}-\d{2}-\d{2})_', 1) AS jour,
-- 2. compter les fichiers distincts (les exports), les lignes et les tickets distincts
       ____
FROM read_csv('sources/caisse/*/tickets_*.csv', filename = ____)
GROUP BY jour
```


© 2026 iAlexMG. Tous droits réservés.

```sql
-- 3. ne garder que les jours exportés plus d'une fois
HAVING ____;
-- Attendu : une seule ligne, 2025-06-12, 2 exports, 123 lignes, 65 tickets.
```

3.3 Comparer les deux exports du 12 juin. `PARSE_FILENAME` garde le nom du fichier sans son chemin.

```sql
-- 1. pour chaque export du 12 juin : son nom, ses tickets, le premier et le dernier
SELECT ____(filename) AS fichier, COUNT(*) AS tickets,
       MIN(horodatage) AS premier, MAX(horodatage) AS dernier
FROM read_csv('sources/caisse/2025-06/tickets_2025-06-12_*.csv', filename = true)
GROUP BY fichier
ORDER BY fichier;
-- Attendu : 58 tickets jusqu'à 15:03:44 dans tickets_2025-06-12_202506121542.csv,
-- et 65 jusqu'à 17:16:04 dans tickets_2025-06-12_202506130705.csv.
```

La caisse a exporté la journée à 15 h 42, café encore ouvert, puis de nouveau le lendemain à 7 h 05. Le second export contient les 58 tickets du premier, plus les 7 de la fin de l'après-midi. Charger les deux compterait le 12 juin deux fois. C'est le premier constat : le dernier export d'un jour fait foi, et le cours 02 en tire la règle.

Étape 4 — Le torréfacteur · Sources

Le torréfacteur garde deux traces. Le journal des lots, un seul fichier, dit pour chaque lot son profil, ses poids et ce que le logiciel a déclaré. Les courbes donnent une lecture des sondes aux 10 secondes, et marquent quatre moments : la charge, le point bas, le premier craquement et la sortie.

4.1 Compter les lots du journal.

```sql
-- 1. les lots du journal, avec le premier et le dernier début
SELECT COUNT(*) AS lots, MIN(debut) AS premier, MAX(debut) AS dernier
FROM read_csv('sources/torrefaction/____');
-- Attendu : 47 lots, du 2025-06-03 07:00:00 au 2025-06-27 07:55:20.
```

4.2 Compter ce que couvrent les courbes.

```sql
-- 1. les fichiers de courbes (les jours), les lots qu'ils couvrent, et leurs lectures
SELECT ____
FROM read_csv('sources/torrefaction/courbes/*/courbe_*.csv', filename = true);
-- Attendu : 8 jours, 47 lots, 3718 lectures.
```

4.3 Lire les moments marqués d'un lot.

```sql
-- 1. les lectures du lot L-2025-0214 qui portent un événement
SELECT lot_id, seconde, temp_grain, evenement
FROM read_csv('sources/torrefaction/courbes/2025-06/courbe_2025-06-03.csv')
```


© 2026 iAlexMG. Tous droits réservés.

```
WHERE lot_id = 'L-2025-0214' AND ____;
-- Attendu : 4 lignes, CHARGE à 0 s, POINT_BAS à 100, PREMIER_CRAQUEMENT à 660, SORTIE à 780.
```

Le journal des lots est un export complet : le 3 juin déjà, il connaît les 47 lots du mois, alors que la courbe de ce jour n'en couvre que 7, et que les autres n'arriveront qu'avec leur jour. C'est le second constat : un lot ne se contrôle que quand sa courbe est arrivée. Les cours 08 et 11 s'en servent.

4.4 Fermer le notebook : fermer l'onglet du navigateur, puis quitter DuckDB dans l'Anaconda Prompt.

```
.quit
```

Attendu : l'invite de l'Anaconda Prompt revient, toujours dans `Projet_Brulerie`. Le notebook « Cours 01 » reste enregistré : il sera là au prochain `duckdb -ui`.

À retenir

- DuckDB lit un fichier là où il est : on regarde avant de charger, et rien ne s'écrit.
- Un type deviné n'est pas une décision : les montants sortent en `DOUBLE`, et leur total peut changer de dernier chiffre d'une exécution à l'autre.
- Deux constats décident du pipeline : le 12 juin a deux exports, et le journal des lots connaît tout le mois avant les courbes.

© 2026 iAlexMG. Tous droits réservés.